

Clase 6 – Actividad

Sistema de control de accesos, alertas y auditoría operativa con Redis

Contexto del caso

Una empresa utiliza un sistema de auditoría automatizada para controlar accesos, validar proveedores y monitorear procesos críticos. El sistema principal de negocio está en una base relacional, pero necesita una capa rápida para responder preguntas operativas en tiempo real.

Redis será utilizado como capa de alta velocidad para resolver situaciones como:

- saber si un usuario está bloqueado;
- contar intentos fallidos de acceso;
- mantener una alerta temporal;
- registrar el estado actual de un proceso automático;
- identificar proveedores observados;
- ordenar proveedores por nivel de riesgo.

La idea no es guardar toda la historia del negocio en Redis, sino usarlo para gestionar **estado actual, reglas rápidas, datos temporales y consultas directas por clave**.

Objetivo de la actividad

Diseñar e implementar en Redis una solución simple que permita:

- guardar estados operativos;
- consultar información por clave;
- agrupar atributos mediante hashes;
- validar pertenencia usando sets;
- usar contadores para eventos;
- aplicar TTL para bloqueos o alertas temporales;
- utilizar un ranking simple con sorted sets.

Situación inicial

El sistema debe manejar los siguientes elementos:

Usuarios:

- usuario_101
- usuario_202
- usuario_303

Proveedores:

- prov_203
- prov_450
- prov_778
- prov_999

Procesos de auditoría:

- proc_1001
- proc_1002

Transacciones:

- trx_9001
- trx_9002
- trx_9003
- trx_9004

Consignas obligatorias

Diseño de claves

Antes de ejecutar comandos, el grupo debe definir una convención de claves.

Por ejemplo:

login:intentos:usuario_101
login:bloqueado:usuario_101
auditoria:proc_1001
proveedores:observados
transacciones:sospechosas
ranking:riesgo:proveedores

La idea central es que la clave sea clara, semántica y fácil de interpretar.

Estados simples con SET, GET, EXISTS y DEL

Crear claves simples para representar estados operativos.

Ejemplo orientativo:

SET sistema:estado "activo"
SET tarea:lote_001 "pendiente"
GET sistema:estado
EXISTS tarea:lote_001

Luego modificar el estado de la tarea:

SET tarea:lote_001 "en_ejecucion"
GET tarea:lote_001

Finalmente eliminar una clave y comprobar que ya no existe:

DEL tarea:lote_001
EXISTS tarea:lote_001

Proceso de auditoría con HASH

Crear un hash para registrar el estado de un proceso de auditoría.

Debe incluir al menos estos campos:

- estado;
- usuario_responsable;
- fecha_inicio;
- resultado;
- cantidad_observaciones.

Ejemplo orientativo:

HSET auditoria:proc_1001 estado "en_ejecucion" usuario_responsable "usuario_101"
fecha_inicio "2026-04-12" resultado "pendiente" cantidad_observaciones 0

Consultar un campo específico:

HGET auditoria:proc_1001 estado

Consultar todo el hash:

HGETALL auditoria:proc_1001

Actualizar el proceso:

HSET auditoria:proc_1001 estado "finalizado" resultado "con_observaciones"
HINCRBY auditoria:proc_1001 cantidad_observaciones 3

Proveedores observados con SET

Crear un conjunto de proveedores observados:

SADD proveedores:observados "prov_203" "prov_450" "prov_778"

Consultar todos los proveedores observados:

SMEMBERS proveedores:observados

Verificar si un proveedor específico está observado:

SISMEMBER proveedores:observados "prov_203"

SISMEMBER proveedores:observados "prov_999"

Agregar un nuevo proveedor y eliminar otro:

SADD proveedores:observados "prov_999"

SREM proveedores:observados "prov_450"

Intentos fallidos y bloqueo temporal

Simular intentos fallidos de acceso para usuario_202.

INCR login:intentos:usuario_202

INCR login:intentos:usuario_202

INCR login:intentos:usuario_202

INCR login:intentos:usuario_202

INCR login:intentos:usuario_202

GET login:intentos:usuario_202

Si el usuario llega a 5 intentos, generar un bloqueo temporal de 10 minutos:

SET login:bloqueado:usuario_202 "1" EX 600

Consultar el bloqueo:

GET login:bloqueado:usuario_202

TTL login:bloqueado:usuario_202

Transacciones sospechosas con SET

Crear un conjunto de transacciones sospechosas:

SADD transacciones:sospechosas "trx_9001" "trx_9003"

Verificar si una transacción está marcada:

SISMEMBER transacciones:sospechosas "trx_9001"

SISMEMBER transacciones:sospechosas "trx_9002"

Ranking de riesgo con Sorted Set

Crear un ranking de proveedores por nivel de riesgo.

ZADD ranking:riesgo:proveedores 95 "prov_203"

ZADD ranking:riesgo:proveedores 70 "prov_450"

ZADD ranking:riesgo:proveedores 88 "prov_778"

ZADD ranking:riesgo:proveedores 60 "prov_999"

Consultar el ranking de mayor a menor riesgo:

ZREVRANGE ranking:riesgo:proveedores 0 -1 WITHSCORES

Consultar los dos proveedores de mayor riesgo:

ZREVRANGE ranking:riesgo:proveedores 0 1 WITHSCORES

Consultar proveedores con score entre 80 y 100:

ZRANGEBYSCORE ranking:riesgo:proveedores 80 100 WITHSCORES

Desafío integrador

El grupo debe implementar este flujo completo:

Un usuario intenta ingresar al sistema. Si acumula 5 intentos fallidos, queda bloqueado por 10 minutos. Si el usuario está bloqueado, el sistema debe rechazar el acceso. Además, si durante el proceso se detecta una transacción sospechosa, debe quedar registrada en un set. Finalmente, el proveedor asociado debe actualizar su score de riesgo.

Este desafío integra los principales conceptos trabajados en Redis. No se busca solamente ejecutar comandos, sino comprender cómo Redis puede actuar como una capa operativa de alta velocidad dentro de un sistema.

El flujo permite aplicar distintos usos reales:

- El contador de intentos representa la acumulación de eventos mediante INCR.
- El bloqueo temporal muestra cómo Redis puede sostener estados transitorios mediante SET con EX.
- La consulta del bloqueo permite ver cómo una aplicación puede decidir rápidamente si permite o rechaza un acceso.
- El set de transacciones sospechosas permite registrar pertenencia sin duplicados y validar si una transacción fue marcada.
- El sorted set de proveedores permite mantener un ranking dinámico de riesgo, actualizando el score cuando ocurre un evento relevante.

La idea es que el grupo pueda interpretar a Redis como una herramienta para responder preguntas operativas inmediatas, por ejemplo: "¿cuántos intentos fallidos lleva este usuario?", "¿está bloqueado ahora?", "¿esta transacción ya fue marcada como sospechosa?" o "¿cómo cambia el nivel de riesgo de un proveedor después de un evento?".

Comandos esperados, a modo orientativo:

INCR login:intentos:usuario_303

GET login:intentos:usuario_303

SET login:bloqueado:usuario_303 "1" EX 600

GET login:bloqueado:usuario_303

TTL login:bloqueado:usuario_303

SADD transacciones:sospechosas "trx_9004"

SISMEMBER transacciones:sospechosas "trx_9004"

ZINCRBY ranking:riesgo:proveedores 10 "prov_999"

ZREVRANGE ranking:riesgo:proveedores 0 -1 WITHSCORES

El grupo debe explicar el flujo en lenguaje natural, no solo pegar comandos y responder:

1. ¿Qué representa la clave login:intentos:usuario_303 y por qué se usa INCR en lugar de SET para actualizarla?
2. ¿Qué significa la clave login:bloqueado:usuario_303 y qué representa el valor "1"?
3. ¿Qué función cumple EX 600 en el bloqueo del usuario? ¿Qué ocurrirá automáticamente cuando pasen los 600 segundos?
4. ¿Por qué transacciones:sospechosas se modela como un set y no como una clave simple?
5. ¿Qué permite resolver SISMEMBER transacciones:sospechosas "trx_9004" dentro del flujo?
6. ¿Por qué el ranking de riesgo de proveedores se modela con un sorted set y no con un set común?
7. ¿Qué rol cumple Redis en este escenario: sistema principal de registro, caché, gestor de estado, motor de reglas rápidas o una combinación de varios? Justificar.
8. ¿Qué información de este caso debería quedar registrada en una base relacional o sistema transaccional, y qué información tiene sentido manejar en Redis?

Entregable del grupo

Cada grupo debe entregar un documento breve con:

1. Nombre del grupo e integrantes.
2. Convención de claves utilizada.

3. Comandos ejecutados, ordenados por sección.
4. Capturas o salidas relevantes de Redis.
5. Respuestas a las preguntas del desafío.
6. Breve conclusión: qué rol cumple Redis en este caso y qué datos no deberían guardarse allí.

Cierre conceptual esperado

La conclusión que debería quedar es que Redis no se usa para reemplazar a la base relacional, sino para resolver necesidades operativas de muy baja latencia.

En este caso, Redis permite:

- responder si un usuario está bloqueado;
- contar intentos fallidos;
- mantener bloqueos temporales;
- validar pertenencia a conjuntos;
- registrar estados de procesos;
- priorizar proveedores por riesgo.

La base relacional seguiría conservando el historial formal de usuarios, transacciones, auditorías y proveedores. Redis, en cambio, sostiene el presente operativo del sistema.

